

Pick-and-Place Simulation

Using a 6-DOF Industrial Robot and RoboDK

MCTR 420 - Introduction to Robotics and Automation

Authors: Isaac Cejas and Totus Tuus Keely

Primary roles: Cejas — simulation development; Keely — Python programming. Both team members contributed to workcell development, motion testing, validation and documentation.

Instructor: Yueh-Jaw Lin

Date: May 6, 2026

Abstract

This project focused on the design, programming and simulation of a six-degree-of-freedom (6-DOF) industrial robot (HIWIN RA605-710-GB) pick-and-place task. The robotic system was programmed and simulated on RoboDK to move between three distinct locations. The simulation emphasized robot kinematics, coordinate frames, homogeneous transformations and motion validation.

The robotic system was fully implemented within the RoboDK simulation software entirely within the RoboDK simulation environment without physical hardware. On the software, multiple coordinate systems were established, including the robot base frame, tool frame and user-defined reference frames. The simulation outputs the values for the homogeneous transformation matrices relative to a specified reference coordinate frame. These values were then validated through hand calculations. The orthogonality and determinant conditions of the rotation matrices were also verified to ensure valid motion.

Introduction

Industrial robots are commonly used in manufacturing and automation for repetitive tasks because of its speed, repeatability and precision. A common implementation for the industrial robots is a pick-and-place motion. Pick-and-place is when a robotic manipulator picks up an object and transfers the object between locations.

The purpose of this project was to design and simulate a robotic pick-and-place task using a HIWIN RA605-710-GB robotic model and the RoboDK simulation software. The simulation involved moving a box between three distinct pedestal locations of varying height with nontrivial orientation changes in three-dimensional space.

The project emphasized key robotic concepts such as coordinate frame definition, robot kinematics, homogeneous transformations, rotation matrices, end-effector orientation control and rigid-body motion validation.

Project Objectives

Multiple objectives were given for this project. First, the simulation environment must be created with a realistic pick-and-place scenario. Next, coordinate reference frames had to be created for the robot base, object and end-effector. The robot then had to have the target positions trained using the software. Once the training was set, the Python program had to be programmed for object interaction and movement. The Python program would then output the homogeneous transformation matrices between key poses to be used for hand-verification. Once complete, the

simulation would demonstrate smooth and realistic robot motions following valid rigid-body transformations.

Robot Model and Coordinate Frame Setup

The model used in our simulation was a HIWIN RA605-710-GB imported from the RoboDK library. The robot contains 6 independent revolute joints which could each be controlled by either a dragging motion or through desired values being inputted such as its x, y, z, or angle with respect to its selected frame which can also be manipulated. Due to the flexibility of the robot, the pick-and-place task was able to be accomplished with a smooth motion which avoided singular configurations or unrealistic motion paths.

Several coordinate frames were defined in the simulation to allow for the robot to be taught specific locations to move towards. User defined coordinate frames were also made for the imported objects such as the base table, pedestals, and box in order to allow functions that could reset the position and orientation of the box to its starting point. The robot base had a fixed coordinate frame in order to act as a reference frame for the position of the end-effector tool. The robot base was also used as a global reference in order for the stationary objects such as the pedestals and table to be positioned and aligned correctly displaying a realistic environment. All motion of the robot was set in reference to the base of the robot so that homogeneous equations could be exported and proven correct through hand calculations. While the orientation and position of the box would have its reference point change between the pedestals and the end-effector depending on which step the simulation was in.

A tool frame was also established for the end-effector, which represented the local coordinate system attached to the gripper used to manipulate the box during the pick-and-place

task. The tool frame allowed for the robot to maintain a correct orientation and position when gripping, lifting, and placing the box on its respective pedestal. Defining the tool frame allowed the box's motion to be observed with respect to the robot base due to the movement being dependent on the pose of the end effector as opposed to the position and orientation of the individual joints of the robot arm.

Pick-and-Place Task Description

The task for our project was to create a realistic pick-and-place simulation while achieving correct requirements such as needing three distinctive end-effector poses. The simulation began with the robotic arm picking up a box from the top face and moving it to a higher and further away pedestal with the top face of the box now facing the robot. After the first placement, the arm would move towards its initial starting point which was labeled as home. From this home position the robot will move towards the second pedestal and attach to the box with the same orientation that it was dropped off with. After the box was attached to the end-effector, the robot tool would keep its orientation fixed as it moved towards the last pedestal mirroring the position of the first pedestal. From here the robot will move back towards its initial position in which the simulation is complete.

The simulation of the robot pick-and-place task realistically displays that of what can be seen in a factory with automation. For example, the initial starting point in which the object is picked up could represent the end of a conveyor belt or chute. From this location the object would then need to be moved to a higher and further away position in order to allow a paint job or attachment of parts to be completed by a different system. After the external system has completed its task with the object, the object would then need to be moved off of its current

position and towards a conveyor belt in which it could go to be packaged. This is one of the many ways in which a task such as a pick-and-place could be utilized in factory automation and depicts how the simulation done with the robotic arm is realistic in a real world setting.

Kinematic Analysis

When it came to the kinematic analysis of the robot, we used Python code to output the homogeneous transfer matrices for the tool movement from home to pedestal 1, pedestal 1 to pedestal 2, and finally pedestal 2 to pedestal 3 as shown in **Figures 1 2 and 3**.

```
Relative Transforms:
Home → P1:
Pose(556.088, -289.768, -389.603, 38.330, 89.440, 51.020):
[[ 0.006, -0.008, 1.000, 556.088 ],
 [ 1.000, 0.011, -0.006, -289.768 ],
 [ -0.011, 1.000, 0.008, -389.603 ],
 [ 0.000, 0.000, 0.000, 1.000 ]]
```

Figure 1. Homogeneous Equation from Home to Pedestal 1

```
P1 → P2:
Pose(234.906, 569.069, -452.804, -88.941, -3.307, -2.646):
[[ 0.997, 0.046, -0.058, 234.906 ],
 [ 0.057, 0.021, 0.998, 569.069 ],
 [ 0.047, -0.999, 0.018, -452.804 ],
 [ 0.000, 0.000, 0.000, 1.000 ]]
```

Figure 2. Homogeneous Equation from Pedestal 1 to Pedestal 2

```
P2 → P3:
Pose(297.007, -291.383, -397.721, -0.828, -0.000, 0.000):
[[ 1.000, -0.000, -0.000, 297.007 ],
 [ 0.000, 1.000, 0.014, -291.383 ],
 [ 0.000, -0.014, 1.000, -397.721 ],
 [ 0.000, 0.000, 0.000, 1.000 ]]
```

Figure 3. Homogeneous Equation from Pedestal 2 to Pedestal 3

It should be noted that due to the software using precise numerical values for computation, values such as 0.006 would be counted as 0 when observing the matrices to make

an inference on how the tool will move from point to point. Orthogonality of the rotation matrices and the determinant condition were hand calculated for one matrix to verify the rest. For the orthogonality calculation in **Figure 4** the values can be approximated to either 0 or 1 due to precision of the software going up to 6 decimal places.

$A = \begin{bmatrix} 0.997 & 0.046 & -0.058 \\ 0.057 & 0.021 & 0.998 \\ 0.047 & -0.999 & 0.018 \end{bmatrix}$
$A^T A = \begin{bmatrix} 0.999467 & 1.06 \times 10^{-4} & -9.4 \times 10^{-5} \\ 1.06 \times 10^{-4} & 1.00056 & 3.08 \times 10^{-4} \\ -9.4 \times 10^{-5} & 3.08 \times 10^{-4} & 0.999692 \end{bmatrix}$

Figure 4. Orthogonality Proof of Rotation Matrix

For the determinant condition, the same concept can be applied in which the value can be approximated to be equal to 1 due to the precision of the software.

$A = \begin{bmatrix} 0.997 & 0.046 & -0.058 \\ 0.057 & 0.021 & 0.998 \\ 0.047 & -0.999 & 0.018 \end{bmatrix}$
$\det(A) = 0.99985828 \approx 1$

Robot Programming and Simulation

The simulation began with a program titled “Replace Objects”. This program allowed for the box’s position and orientation to be reset to the first pedestal. This simplified the experimentation process so that if the box was ever incorrectly moved, attached, or deleted it could be moved back to its original starting point. The “Home” program then followed which essentially played the same role as the previous program but instead of resetting the box, the

position of the robot arm was brought back to its initial starting point. This also simplified the trial and error process of programming as whenever the robot moved in an unexpected position, we were able to restart without needing to manually reset the environment.

After the initialization was complete, the program titled “Initial” would be called. This program would have the robot arm move to a position above the box with the end-effector tool oriented to be ready to pick up the box from its sides. The purpose of having the tool frame sit above the box rather than going straight to attaching to it was to prevent any collision between the box and the tool. Next up was the program titled “Go to P1” in which the tool drops down and picks up the box moving it towards the pedestal one. The feature event instruction allowed for an "attach object” instruction to be placed between the box and tool which allowed for both to move in unison. The program “Back Home” utilized the event instruction "detach object” to disconnect the attachment between the tool and the box and then proceeded to move the robot back to its initial position. The reason behind the back home movement was to realistically simulate how the robot arm would return home while it waits for the object to be painted or worked on.

After the second pedestal the program then calls “Go to P3” in which the robot moves towards the box with the tool being in the same orientation as it dropped the box off in. After attaching to the box again, the tool orientation is kept the same as the robot proceeds to move towards the third and final pedestal. Once the box is placed on the third and final pedestal, the program “Back Home Again” is called which has the same function as the “Back Home” program in which the box and robot are detached from one another and the arm is moved back to its home position. Python code was attached to the workflow of the simulation to allow for the homogeneous equations to be produced in order to validate the movement of where the

end-effector should be through hand calculations.

Motion Validation and Visualization

The simulation displayed correct movement as the robot tool never had any unnecessary collisions with the pedestal or box throughout the entire pick-and-place task. The box was successfully picked up and placed between pedestals as seen in Figure 1, Figure 2, and Figure 3.

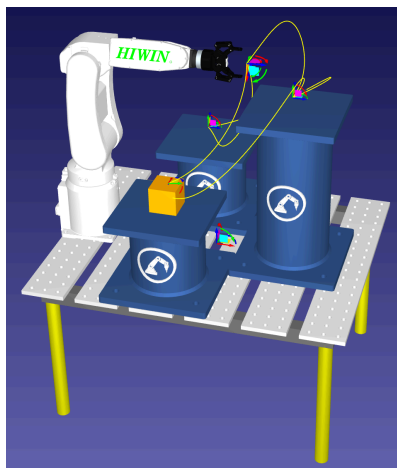


Figure 1. Location 1

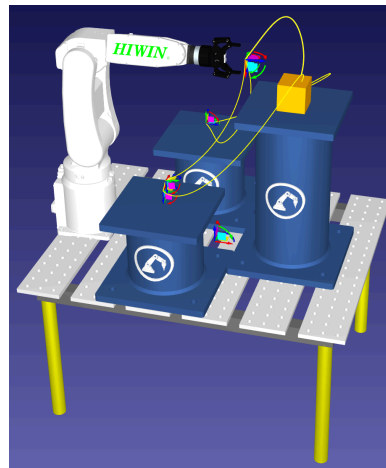


Figure 2. Location 2

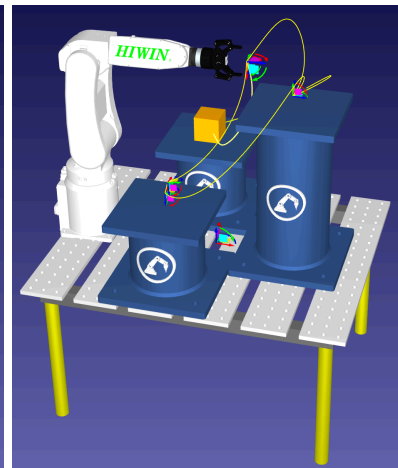


Figure 3. Location 3

Engineering Discussion

Rotation matrices used in robotics must be orthonormal because most robots are rigid-body systems. If a non-unitary rotation matrix were used, the transformation could introduce physically impossible actions such as scaling or skewing.

Incorrect transformations could lead to an improper end-effector orientation, position inaccuracies, potential collisions and/or damage to equipment or workpieces. Small numerical errors could lead to incorrect transformations. If the computer system rounds or truncates

numbers at a low precision, the errors could compound at each joint movement of the robot leading to a large deviation at the end-effector. The precision of a robot is an often reported specification and should be verified to be enough for the task.

One challenge encountered during the project was ensuring there were not any collisions in the pathway. Due to the robot using joint movements, the shortest path, from a target point to the next, the robot would often move itself or the object into a collision with the pedestal. This issue was countered by adding another target point leading the robot away from the collision to ensure the path was clear.

Conclusions

This project successfully demonstrated the programming and simulation of a 6-DOF industrial robot performing a pick-and-place task using RoboDK and Python. The HIWIN RA605-710-GB robot was able to move objects between multiple locations while maintaining proper positioning and orientation.

Important robotic concepts including coordinate frames, homogeneous transformations and robot kinematics were applied throughout the project. The generated transformation matrices were verified through hand calculations, including orthogonality and determinant checks for the rotation matrices. Overall, the project provided practical experience in robotic simulation, motion programming and rigid body transformation analysis within a virtual industrial automation environment.

References

[1] RoboDK Inc., “RoboDK Documentation and Simulation Software,” RoboDK, 2026.

[Online]. Available: <https://robodk.com>

[2] Y.-J. Lin, MCTR 420 - Introduction to Robotics and Automation Course Notes and Project Guidelines, 2026.

Appendices

Appendix A - Hand-Verification of Transformation Matrices

$A = \begin{bmatrix} 0 & 0 & 1 & 774.5 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 380 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	
$B = \begin{bmatrix} .006 & -.008 & 1 & 556.088 \\ 1 & 0.011 & -.006 & -289.768 \\ -0.011 & 1 & 0.008 & -389.603 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	
AB	$= \begin{bmatrix} -0.011 & 1 & 0.008 & 384.897 \\ 1 & 0.011 & -0.006 & -289.768 \\ -0.006 & 0.008 & -1 & -176.088 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad \text{Ⓢ}$

Figure A1: Verification of the transformation matrix from home (matrix A) to the first pedestal (matrix B)

$$\begin{aligned}
A &= \begin{bmatrix} -.011 & 1 & 0.008 & 384.897 \\ 1 & 0.011 & -.006 & -289.768 \\ -.006 & 0.008 & -1 & -176.088 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
B &= \begin{bmatrix} .997 & 0.046 & -.058 & 234.906 \\ 0.057 & 0.021 & .998 & 569.069 \\ 0.047 & -.999 & 0.018 & -452.804 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
AB &= \begin{bmatrix} 0.046409 & 0.012502 & 0.998782 & 947.76 \\ 0.997345 & 0.052225 & -0.04713 & -45.8854 \\ -0.052526 & 0.998892 & -0.009668 & 279.859 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad \text{Ⓢ}
\end{aligned}$$

Figure A2: Verification of the transformation matrix from pedestal 1 (matrix A) to the second pedestal (matrix B).

$$\begin{aligned}
A &= \begin{bmatrix} .046 & .013 & .999 & 947.782 \\ .998 & .052 & -.046 & -45.668 \\ -.053 & .999 & -.011 & 279.573 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
B &= \begin{bmatrix} 1 & 0 & 0 & 297.007 \\ 0 & 1 & .014 & -291.383 \\ 0 & -.014 & 1 & -397.721 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
AB &= \begin{bmatrix} 0.046 & -9.86 \times 10^{-4} & 0.999182 & 560.333 \\ 0.998 & 0.052644 & -0.045272 & 253.888 \\ -0.053 & 0.999154 & 0.002986 & -22.8851 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad \text{Ⓢ}
\end{aligned}$$

Figure A3: Verification of the transformation matrix from pedestal 2 (matrix A) to the third pedestal (matrix B)